

Move Less, Retrieve Fast: A Retrieval-in-Memory Architecture for Language Models

Jiaxian Chen¹, Yuxuan Qi¹, Jianan Yuan¹, Kaoyi Sun¹, Tianyu Wang¹, Chenlin Ma¹ and Yi Wang^{1,2}

1. College of Computer Science and Software Engineering, Shenzhen University

2. State Key Laboratory of Radio Frequency Heterogeneous Integration

Abstract—Retrieval-augmented language models (RALMs) have attracted widespread attention for addressing the limitations of traditional large language models. However, challenges involved in retrieval, including substantial data movement and irregular access patterns, seriously impact the efficiency and deployment of RALMs. The emerging 3D-stacked processing-in-memory (PIM) architecture, characterized by its high memory bandwidth and near-data computing capabilities, presents a promising solution for efficient retrieval. To support large-scale retrieval in RALMs, the PIM architecture should be carefully designed with joint software and hardware optimization.

This paper presents *Rimast*, a retrieval-in-memory architecture for fast retrieval in RALMs. The objective is to minimize data movement and improve overall performance through hardware-software co-design. At the hardware level, a hierarchical PIM architecture with a retrieval-in-memory dataflow is designed to reduce unnecessary data transfer. At the software level, skew-free data mapping and adaptive offloading strategies are proposed to address the irregular access patterns associated with retrieval in RALMs. We demonstrate the effectiveness of the proposed Rimast using extensive experiments. The experimental results demonstrate that Rimast effectively reduces data movement, achieving average speedups of 273 $\times$, 55 $\times$, and 2.41 $\times$ over CPUs, GPUs, and prior art accelerators, respectively.

Index Terms—Retrieval, RALM, Processing-in-Memory

I. INTRODUCTION

Retrieval-augmented language models (RALMs) [5], [22] have been successfully deployed in various application domains. RALMs combine non-parametric external storage with internal parameters, and this hybrid approach provides better performance than traditional large language models [9], [10]. RALMs typically employ a large-scale external database with billions of embeddings to enhance language models. RALMs suffer from low efficiency caused by large-scale retrieval from external database, making them 12 $\times$ slower than traditional language models (LMs) [27].

Fig. 1(a) illustrates the workflow of a typical RALM, which consists of three major steps: (i) *BERT*; (ii) *Retrieval*; and (iii) *LM*. Upon receiving an input context, it is transformed into chunk embeddings using BERT [8]. These embeddings query a key-value database to retrieve the most similar text chunks, which are then integrated into LMs to generate high-quality responses. Although RALMs greatly enhance response accuracy, *retrieval* (the second step) accounts for 93.8% of the processing time in the RALMs pipeline, as illustrated in Fig. 1(b), highlighting the significant potential for optimization.

Efficient retrieval approaches mainly comprise graph-based, hashing-based, and compression-based algorithms. Although

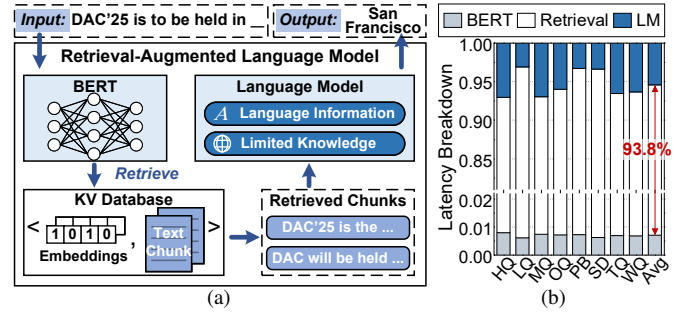


Fig. 1: (a) RALM workflow. (b) Latency breakdown.

graph-based [17] and hashing-based [26] algorithms perform well with high accuracy during retrieval, they often require a high memory footprint of up to several terabytes of memory in RALMs [11]. In contrast, compression-based retrieval algorithms can achieve high performance and low memory footprint [13]. Many RALMs adopt compression-based algorithms for large-scale retrieval [5], [20], [23], [25]. However, retrieval in RALMs still involves substantial data movement that algorithm optimization can not fully address. Numerous embeddings in external databases will be transferred from memory to processors, which causes significant performance degradation and energy consumption.

The processing-in-memory (PIM) architecture [6], [7], [21], [28] provides a promising solution for RALMs. The advanced 3D memory [14] stacks multiple DRAM dies on a base die, offering significant capacity and bandwidth for RALMs. Furthermore, computing units can be placed in DRAM dies to enjoy *in-memory retrieval*, effectively reducing data movement. However, processing retrieval efficiently via the 3D-stacked PIM architecture is non-trivial. First, the existing centralized processing dataflow is not suitable for in-memory retrieval. Second, irregular access issues in RALMs lead to imbalanced workloads, limiting the benefits of PIM architectures.

This paper proposes *Rimast*, a retrieval-in-memory architecture for fast billion-scale retrieval in RALMs. The proposed Rimast minimizes data movement and improves the overall performance for retrieval using a hardware-software co-design approach. At the hardware level, a hierarchical PIM architecture with a retrieval-in-memory dataflow is designed to minimize unnecessary data transfer. At the software level, a skew-free data mapping strategy is presented to achieve fine-grained data partitioning and even data distribution. An adaptive offloading strategy is further proposed to reallocate tasks at runtime, eliminating imbalanced workloads. We demonstrate the effec-

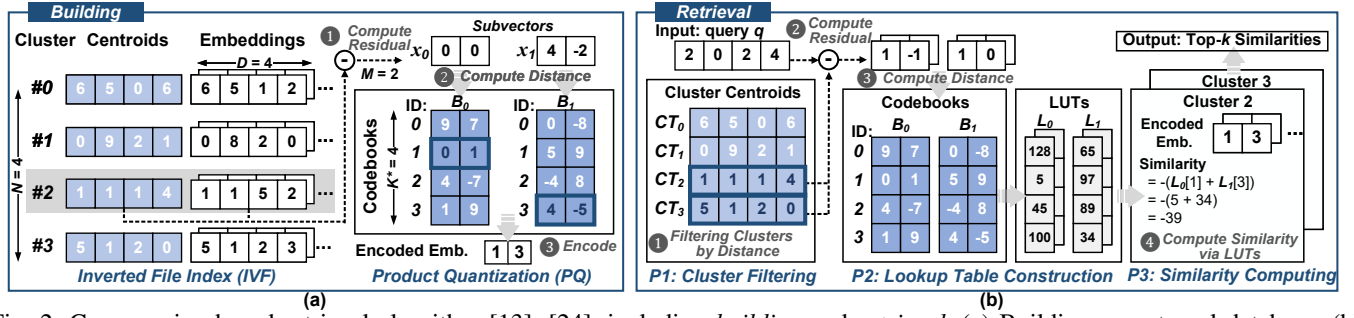


Fig. 2: Compression-based retrieval algorithm [13], [24], including *building* and *retrieval*. (a) Building an external database. (b) Three phases of retrieval.

tiveness of Rimast using extensive experiments. Experimental results show that Rimast outperforms existing representative schemes, substantially reducing processing latency and energy consumption.

The main contributions of this work include:

- **PIM for RALM Retrieval:** To the best of our knowledge, this is the first PIM-based retrieval accelerator to address the data movement and the irregular access issues in RALMs.
- **Architecture Design:** A retrieval-in-memory architecture with customized dataflow is designed to minimize off-chip data movement.
- **Software Optimization:** A skew-free data mapping strategy and an adaptive offloading strategy are proposed to address the imbalanced workloads caused by irregular accesses.

The rest of this paper is organized as follows: Section II illustrates the background and motivation, Section III details the proposed techniques, and Section IV presents and discusses the experimental results. Finally, Section V concludes the paper.

II. BACKGROUND

A. Large-Scale Embedding-Based Retrieval

Embedding-based retrieval aims to identify the top- k embeddings in a database that are most similar to a given query embedding. With the L2 (Euclidean) distance, the similarity between the query q and an embedding y is defined as:

$$S(q, y) = -\|q - y\|_2^2 = -\|q\|^2 - \|y\|^2 + 2\langle q, y \rangle \quad (1)$$

An exact search over a large-scale database is computationally expensive, often requiring significant time and memory.

To alleviate the overhead, compression-based retrieval algorithms are widely adopted for their high performance and low memory footprint. Compression-based algorithms primarily adopt inverted file (IVF) index and product quantization (PQ) [13]. The IVF index divides embeddings into N clusters, each represented by a centroid. It significantly reduces computation by restricting similarity computations to the selected clusters. PQ reduces storage and computation costs by mapping high-dimensional embeddings to lower-dimensional spaces. It achieves this by dividing a D -dimensional embedding into M subvectors, each of which is quantized using a learned codebook with K^* codewords. An example of building a database using IVF and PQ is illustrated in Fig. 2(a). The retrieval process

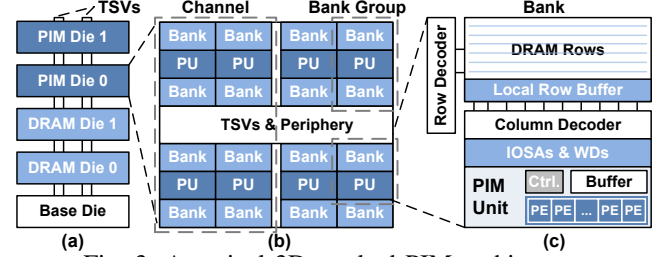


Fig. 3: A typical 3D stacked PIM architecture.

consists of three main phases: (1) *cluster filtering*, (2) *lookup table (LUT) construction*, and (3) *similarity computing*. An example of retrieval is depicted in Fig. 2(b). Given a query embedding $q = \|2 \ 0 \ 2 \ 4\|$, the process starts by selecting candidate clusters with the most similar centroids (i.e., CT_2 and CT_3). Next, LUTs are built to reduce redundant computations, as each encoded embedding may comprise repeated codewords. For instance, the residual embedding of q relative to CT_2 is split into two segments and compared with codewords in B_0 and B_1 to form L_0 and L_1 . Finally, similarities between q and encoded embeddings are efficiently computed via LUTs.

B. 3D-Stacked Processing-in-Memory Architecture

Fig. 3 illustrates a typical processing-in-memory (PIM) architecture utilizing 3D stacked memory [14], where multiple DRAM dies are stacked on top of a base die using through-silicon vias (TSVs). DRAM dies equipped with PIM capabilities are termed PIM dies. As shown in Fig. 3(b), each PIM die contains multiple memory channels, with each channel comprising bank groups and each bank group comprising banks. Fig. 3(c) shows the structure of a DRAM bank with a PIM unit, which includes SRAM buffers and processing elements for near-bank computing. Each bank contains DRAM cells, a global row buffer of I/O sense amplifiers (IOSAs), address decoders, and write drivers (WDs). DRAM cells are arranged in rows and columns. Before accessing memory, the relevant row should be activated. As only one row per bank can be active at a time, accessing two different rows consecutively causes a row buffer conflict, resulting in low bandwidth utilization.

C. Motivation

1) **Data Movement Issue:** Compression-based retrieval involves substantial movement of embeddings from memory to processors, making it memory-bound. We analyze the memory bandwidth requirement of retrieval with the well-established

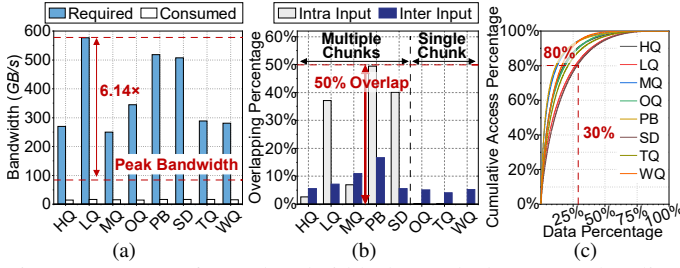


Fig. 4: (a) Up to $6.14\times$ bandwidth demand. (b) Access locality pattern with up to 50% data overlap. (c) Skewed access pattern with 30% data contributing 80% access.

library Faiss [12]. Fig. 4(a) indicates that retrieval performance is constrained by limited memory bandwidth. To match the throughput of inference, the system requires a higher bandwidth, ranging from $2.65\times$ to $6.14\times$ the peak memory bandwidth of a four-channel system. Since the memory-bounded similarity computing phase mainly involves addition operations, it can be efficiently processed in memory with low hardware overhead.

2) **Irregular Access Issue:** The workloads of RALMs exhibit irregular access patterns. First, RALMs exhibit strong temporal locality. A long input is divided into chunks, generating several query chunk embeddings. Adjacent chunk embeddings within an input often access the same clusters, with up to 50% overlap (Fig. 4(b)). Second, retrieval has a skewed access pattern. 30% of the clusters account for over 80% of the accesses, while others receive minimal attention (Fig. 4(c)). These access patterns cause imbalanced workloads, especially in PIM architectures with in-place data processing.

Motivated by the above-mentioned challenges, we focus on exploiting PIM architectures for retrieval via a hardware and software co-design approach.

III. RIMAST: AN RETRIEVAL-IN-MEMORY ARCHITECTURE FOR RALMS

Rimast is a HW/SW co-design architecture tailored for retrieval in RALMs. At the hardware level, a hierarchical PIM architecture with a customized dataflow is designed to exploit in-memory retrieval. At the software level, Rimast applies two strategies to address irregular access patterns. First, *skew-free data mapping* is proposed to distribute embeddings evenly, avoiding imbalanced workloads caused by the skewed pattern. Second, *adaptive offloading* is proposed to reduce latency by rescheduling tasks with temporal locality patterns.

A. Hardware Architecture Design

1) **Overview:** The overall hardware architecture of Rimast is demonstrated in Fig. 5. Rimast comprises a centralized engine (CE) and multiple memory groups, where a chain topology [18] is adopted to connect them. Each memory group integrates multiple memory stacks and a control chip. The control chip contains memory controllers (MCs), an internal switch, and a router for off-chip communication. The CE constructs LUTs via a *LUT construction unit (CU)*, schedules retrieval tasks, and aggregates similarity results, while PIM-enabled memories process in-memory retrieval. Rimast adds dedicated logic to multiple levels to accelerate retrieval jointly. At the base die

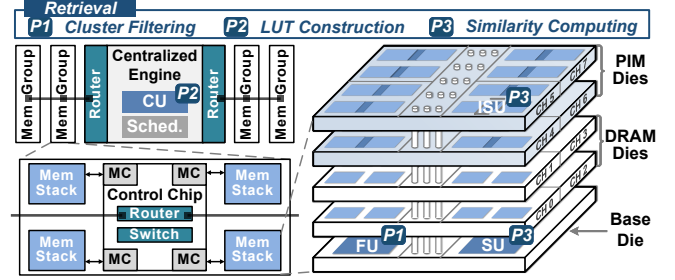


Fig. 5: Architecture design overview. Dedicated compute units (CU, FU, SU, and ISU) are added for retrieval.

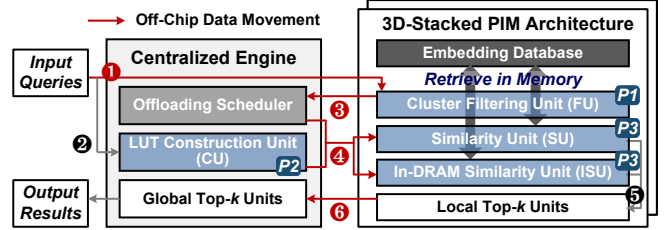


Fig. 6: Retrieval-in-memory dataflow.

level, the *cluster filtering unit (FU)* is customized to select candidate clusters, and the *similarity unit (SU)* is designed to compute similarities for overlapping accessed clusters. At the PIM die level, the *in-DRAM similarity unit (ISU)* is integrated into each memory bank group to enjoy near-bank processing.

2) **Retrieval-in-Memory Dataflow:** A retrieval-in-memory dataflow is proposed to perform similarity computations directly in memory, transferring only reduced intermediate results rather than the entire database. Fig. 6 presents the proposed dataflow. Once queries come, query embeddings are broadcast to PIM-enabled memories (step 1) and simultaneously sent to CU (step 2). Then, FUs within the PIM memories filter candidate clusters, sending the filtered results to the offloading scheduler in the CE (step 3). Next, the shared partial LUTs are broadcast to all PIM memories, and the scheduler distributes tasks to the PIM-enabled memories (step 4), enabling similarity computing. Finally, local top- k units within each memory stack reduce these similarities (step 5), and the global top- k unit in the CE aggregates the reduced similarities (step 6), completing the retrieval process. Throughout this process, only steps 1 3 4 6 involve minimal off-chip data movement, with lightweight intermediate data and scheduling information being transferred.

The retrieval dataflow is carefully designed with a two-step LUT construction method to reduce data movement between the CE and PIM architectures while avoiding adding extra multipliers to PIM units. The CE handles shared portions of LUTs, and PIM units perform partial accumulations. Given a query q , a cluster centroid y_C , and a valid codebook keyword combination y_B , the L2 distance is decomposed in Equation (2). Here, *term 1* is computed during cluster filtering, and *term 2* is precomputed and stored as it is independent of q . The CE computes *term 3* to form partial LUTs. The complete LUTs are built on demand by accumulating these terms during the similarity computing phase.

$$\|q - y_C - y_B\|_2^2 = \underbrace{\|q - y_C\|_2^2}_{\text{term 1}} + \underbrace{\|y_B\|_2^2}_{\text{term 2}} + 2\langle y_C, y_B \rangle - 2\langle q, y_B \rangle \quad (2)$$

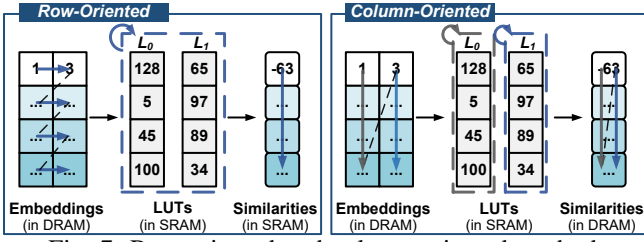


Fig. 7: Row-oriented and column-oriented methods.

3) Detailed Hardware Design:

In-DRAM Similarity Unit (ISU): The ISU is designed for each bank group to support in-memory retrieval, with an optimized embedding processing method to reduce on-chip buffer requirements. Fig. 7 illustrates the conventional row-oriented method and the proposed column-oriented method. The row-oriented method processes encoded embeddings individually, calculating and storing each similarity result one by one. This approach requires repeated random lookups to all LUTs (i.e., L_0 and L_1), necessitating a large on-chip buffer to cache these tables. In contrast, the column-oriented method processes embeddings column by column, accumulating intermediate similarity results sequentially. This approach aligns with the sequential access nature of memory banks to reduce row buffer conflicts. Moreover, it can reduce on-chip buffer demand by multiples as LUTs are accessed individually.

The proposed ISU adopts the column-oriented method. Fig. 8 presents the detailed architecture design of an ISU. The ISU mainly consists of adder arrays, a LUT buffer, a task queue, and a controller. Each task, representing similarity computations for specific clusters, is received by the pending task queue from the offloading scheduler. During similarity computing, the ISU schedules a task (step ①) and constructs LUTs by accumulating the partial LUTs (step ②). To reduce row buffer conflicts, destination and source banks are used to separate the lookup and accumulating operations. Once the LUTs are prepared, embeddings are read in column-oriented order from a source memory bank to the ISU, performing lookup operations (step ③). Next, adder arrays conduct element-wise addition using distances obtained from LUTs, and the resulting partial similarities are accumulated to destination banks (step ④).

Similarity Unit (SU): The locality access pattern of RALMs decides that adjacent query embeddings will have overlapping access clusters, causing repeated access and processing of encoded embeddings. To address the imbalance, Rimast offloads these overlapping clusters from DRAM dies to SUs.

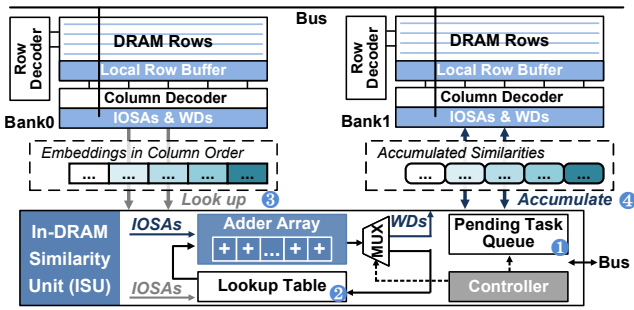


Fig. 8: Hardware design of an in-DRAM similarity unit.

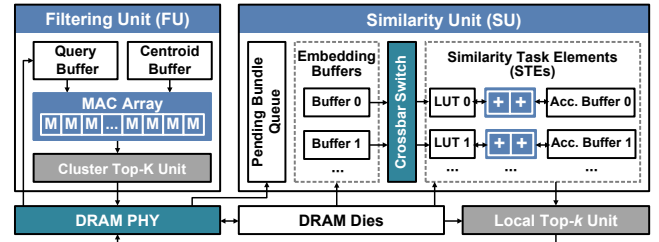


Fig. 9: Hardware design of base die logic.

As shown in Fig.9, an SU includes a pending bundle queue, embedding buffers, a crossbar switch, and similarity task elements (STEs). The pending bundle queue receives task bundles from the offloading scheduler, where each bundle contains multiple tasks targeting the same cluster. First, the control unit allocates embedding buffers and STEs when a bundle is scheduled. Then, the candidate embeddings are fetched from local memory to the buffers, and further broadcast through the crossbar switch to the STEs for similarity computation. Since tasks in a bundle access the same embeddings, STEs can reuse them, reducing redundant data fetching.

Other Compute Units: The FU and CU are designed for cluster filtering and LUT construction, respectively. Both utilize a similar design, employing dot-product operations optimized through MAC arrays for distance calculations.

B. Skew-Free Data Mapping

Due to the skewed access pattern, in-memory retrieval will incur imbalanced workloads. To address this, we propose skew-free data mapping, which distributes embeddings evenly across memories. The proposed mapping is illustrated in Algorithm 1. First, clusters are partitioned into fine-grained sub-clusters, each containing a fixed number of embeddings (Line 1). This approach avoids memory overloading or under-utilization resulting from directly allocating entire clusters to memory banks, as clusters have varying numbers of embeddings. Next, sub-clusters are interleaved across memory banks using an address-mapping strategy (Lines 2-6). The memory hierarchy comprises stacks, channels, bank groups, and banks. The even distribution of sub-clusters across this hierarchy can maximize

Algorithm 1 Skew-Free Data Mapping

Require: Embedding cluster set C , and sub-cluster size sz .
Ensure: Sub-cluster set SC , and mapping set M .

- 1: $SC \leftarrow$ Partition each cluster $C_m \in C$ into sub-clusters with fixed size sz .
- 2: **for** each $SC_n \in SC$ **do**
- 3: $id \leftarrow$ Binary identifier of SC_n
- 4: $pos \leftarrow$ Map id using sub-cluster interleaved mapping
- 5: $PUSH(M, pos)$
- 6: **end for**
- 7: **return** SC and M

Sub-Cluster ID										Sub-Cluster Offset									
Row	Bank	Group	Channel	Stack	Row	Bank	Group	Channel	Stack	Row	Bank	Group	Channel	Stack	Row	Bank	Group	Channel	Stack
...	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

Fig. 10: Sub-cluster interleaved address mapping.

parallelism. Fig. 10 illustrates how the sub-cluster interleaved address mapping allocates each sub-cluster to a specific bank. Given a sub-cluster with an identifier of 1983, represented as $(01\ 111\ 0111\ 111)_2$, it is mapped to memory bank 1, within bank group 7, channel 7, and stack 7.

C. Adaptive Offloading

Rimast offloads overlapping accessed clusters from bank groups to the SU, avoiding workload imbalances caused by the access locality pattern. An adaptive offloading strategy based on integer linear programming (ILP) is proposed to achieve an optimal solution, minimizing overall processing time.

Within each stack, offloading tasks from a bank group BG_i^j in channel CH_i to the SU reduces the processing time of BG_i^j but increases the processing time (or data transfer time) of channel CH_i and the processing time of the SU. The processing time of each component, including BG_i^j , CH_i , and the SU, can be estimated using linear functions and constrained by total processing time. These linear constraints can be used to formulate an ILP model to determine the number of offloading tasks. Given the initial allocation obtained via the skew-free mapping, we formulate the ILP model as follows:

$$\min \quad \mathbb{T} \quad (3)$$

$$\text{s.t.} \quad \sum_i \sum_j x_i^j = x_s, \quad (4)$$

$$\forall i, j : T_{BG_i^j} = \left(\frac{M_{BG_i^j} - x_i^j}{CAP_{BG}} + \frac{OF_i^j(x_i^j)}{BW_{BG}} \right) \leq \mathbb{T}, \quad (5)$$

$$\forall i : T_{CH_i} = \left(\frac{\sum_j OF_i^j(x_i^j)}{BW_{CH}} \right) \leq \mathbb{T}, \quad (6)$$

$$T_{SU} = \left(\frac{x_s}{CAP_{SU}} \right) \leq \mathbb{T}. \quad (7)$$

The objective of the ILP, as expressed in Equation (3), is to minimize the total processing time $\mathbb{T}$. Notations used in our ILP model are summarized in Table I. To ensure balanced workload distribution, constraints are imposed to keep the processing times of all components within $\mathbb{T}$ (Equations (5)-(7)). The ILP determines the number of offloaded tasks for each bank group. Then, sub-clusters with the highest numbers of repeated accesses within each bank group are prioritized for offloading.

IV. EVALUATION

A. Experimental Setup

Workloads. We construct the RALM system using a database with one billion embeddings sampled from the Pile [4] and the large GPT model [2]. Text chunks of 64 tokens are transformed into 384-dimensional embeddings using the all-MiniLM-L12-v2 model [19]. The index structure is built with the IVFPQ

TABLE I: Notations of the ILP model.

Symbol	Description
$T_{SU}, T_{CH_i}, T_{BG_i^j}$	Processing time of SU, CH_i , and BG_i^j
$M_{BG_i^j}$	Number of tasks initially mapped to BG_i^j
OF_i^j	Linear function estimating data size offloaded for BG_i^j
x_i^j	Number of tasks offloaded from BG_i^j
x_s	Number of tasks offloaded to SU
CAP_{BG}, CAP_{SU}	Computing capacities of a bank group and an SU
BW_{BG}, BW_{CH}	Bandwidths of a bank group and a channel

TABLE II: System configurations and power-area models.

PIM System Organization	8 stacks $\times$ 16 channels $\times$ (2 bank groups + 6 PIM bank groups) $\times$ 4 banks, 12GB per stack, 256GB/s off-chip bandwidth per bidirectional link.
Timing (ns)	tRCD=10, tRC=25, tCA =14, tCCD_S=2, tCCD_L=4
CE	384KB buffers, 128 MACs (1 GHz), 2 top-256 units and 2 top-16 units (1 GHz), Power=375.02mW, Area=1.27 mm ²
ISU	2KB dedicated buffers, 48 adders (125 MHz), Power=6.29mW, Area=0.10 mm ² (2.7% of a bank group)
Base die logic	1.25MB buffers, 128 MACs and 512 adders (1 GHz), 1 top-256 unit and 16 top-16 units (1 GHz), Power=1432.21mW, Area=9.48 mm ² (14% of the base die)

algorithm implemented in Faiss [12]. In our experiments, we set the number of clusters $N=16,384$, the number of codewords $K^*=256$, the number of split chunks $M=48$, and the batch size $BS=16$. The candidate cluster count is 256, achieving an average recall rate at 90%. Eight tasks from the LM-Eval [3] benchmark, including HeadQA (HQ), LogicQA (LQ), MathQA (MQ), OpenbookQA (OQ), PubmedQA (PB), SQuAD2 (SQ), TriviaQA (TQ), and WebQs (WQ), are used as input.

Simulation and Configuration. To evaluate our approach, we extend DRAMSim3 [16] to support PIM operations for retrieval. We measure power, area, and delay for each component and integrate these parameters into our simulator. Computing logic is implemented in Verilog and synthesized with the Synopsys Design Compiler using a 32nm library. Scratchpad memories, interconnections, and the 3D DRAM are estimated using CACTI [1]. Detailed configurations are listed in Table II.

Baselines. We compare Rimast with Faiss [12], ANNA [15], and Rimast w/o SU. Faiss provides efficient retrieval solutions for CPU and GPU, with FaissCPU running on 256GB DDR4 memory and two Intel Xeon Silver 4210R CPUs, and FaissGPU running on two NVIDIA A100 (40GB) GPUs. ANNA [15] is a specialized compression-based retrieval architecture. Rimast w/o SU can demonstrate the effectiveness of hierarchical PIM architecture. ANNA and Rimast utilize equivalent computing resources for fair comparison.

B. Experimental Results

1) Throughput: Fig. 11 illustrates the throughput and latency across different baselines. Rimast demonstrates the highest throughput and lowest latency among all baselines. Compared to Rimast w/o SU, ANNA, FaissGPU, and FaissCPU, Rimast achieves average throughput improvements of 1.63 $\times$, 2.41 $\times$, 55.86 $\times$, and 273.78 $\times$, respectively, while reducing latency

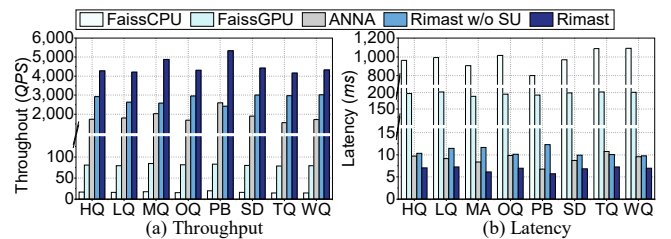


Fig. 11: Throughput and latency.

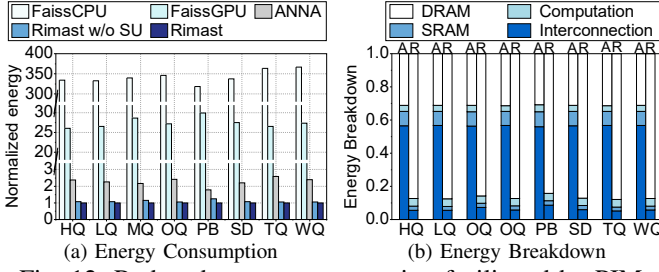


Fig. 12: Reduced energy consumption facilitated by PIM.

by 73.73%, 25.77%, 96.61%, and 99.31%. This performance boost is attributed to the PIM architecture, which significantly minimizes data movement. Moreover, the adaptive offloading mitigates workload imbalances, reducing retrieval latency.

2) *Energy Consumption*: Energy consumption can reflect the data movement reduction due to PIM. Fig.12(a) presents the normalized energy consumption across various benchmarks, showing that Rimast has the lowest average energy consumption. Rimast achieves average energy savings of 1.10 $\times$, 2.27 $\times$, 27.69 $\times$, and 343.85 $\times$ compared to Rimast w/o SU, ANNA, FaissGPU, and FaissCPU, respectively. This is primarily due to adopting the retrieval-in-memory dataflow, which minimizes off-chip data movement and reduces energy consumption. Fig.12(b) provides an energy breakdown for ANNA and Rimast, labeled as “A” and “R” respectively. Unlike ANNA’s centralized dataflow, Rimast’s retrieval-in-memory dataflow significantly lowers the energy consumption of interconnection.

3) *Bandwidth Consumption and Scalability*: The off-chip bandwidth consumption can reflect data movement reduction due to PIM. Fig. 13(a) shows off-chip bandwidth consumption for ANNA and Rimast with different memory stacks, where 8, 16, and 32 stacks consume 1.04 GB/s, 3.91 GB/s, and 13.91 GB/s, respectively. The data movement overhead of Rimast is neglected since it only transfers a small amount of intermediate data like LUTs and top-k results. We also demonstrate the scalability of Rimast by varying memory stack counts. Fig. 13(b) shows normalized throughput and latency changes as stack counts increase. Rimast with 32 memory stacks achieves 3.41 $\times$ and 1.81 $\times$ higher throughput compared to 8 and 16 stacks, respectively, while latency decreases from 6.9 ms (8 stacks) to 3.6 ms (16 stacks) and 2.0 ms (32 stacks). This linear improvement highlights the scalability of Rimast.

4) *Performance under Different Recall Rates*: Fig. 14 compares the throughput and latency of baseline schemes at various recall rates. Across different recall rates, Rimast consistently achieves the highest throughput and lowest latency. Compared to the state-of-the-art compression-based retrieval

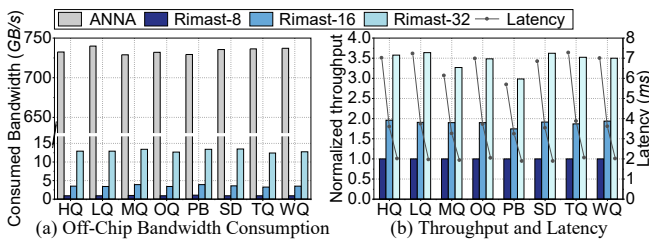


Fig. 13: Scalability of Rimast with increasing memory stacks

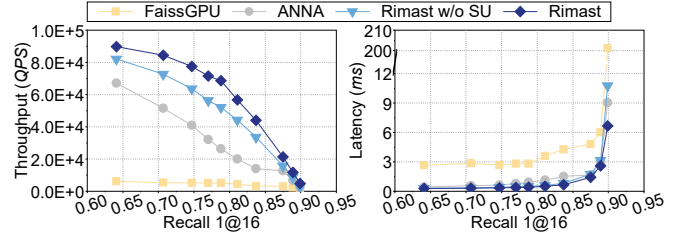


Fig. 14: Throughput and latency under different recall.

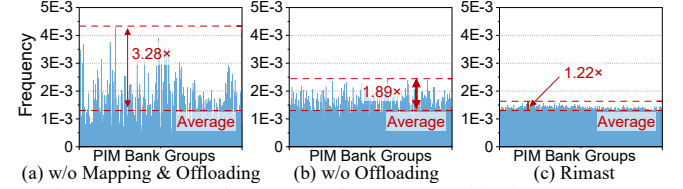


Fig. 15: Access frequency for PIM-enable bank groups.

accelerator, ANNA, Rimast improves throughput by 1.33 $\times$ -3.14 $\times$ and reduces latency by 1.35 $\times$ -2.36 $\times$ on average for recall rates between 60% and 90%. These results highlight the robustness of our method.

5) *Workload Balancing*: Fig. 15 illustrates the memory access frequency distribution across each PIM-enabled bank group. Workload balancing is quantified using the peak-to-average access frequency (PAF) ratio. The adoption of skew-free mapping reduces the PAF from 3.28 $\times$ to 1.89 $\times$, effectively mitigating the skewed access pattern. Further improvement is achieved through adaptive offloading, which reduces the PAF to 1.22 $\times$, addressing imbalances caused by locality patterns.

V. CONCLUSION

This paper presents Rimast, a PIM-enabled architecture for retrieval in RALMs based on 3D-stacked memories. Both hardware and software are carefully co-designed to fully exploit in-memory retrieval. A hierarchical architecture with an optimized dataflow is proposed to minimize data movement and reduce overall processing latency. We also present two optimization strategies, skew-free data mapping and adaptive offloading scheduling to address the irregular access patterns associated with retrieval in RALMs. The experimental results show that Rimast achieves a substantial reduction in processing latency and energy consumption.

ACKNOWLEDGEMENTS

The work described in this paper is supported in part by NSFC (U23B2040), in part by National Key Research and Development Program of China (2024YFB4505702), in part by Shenzhen Science and Technology Program (RCJC20221008092725019), in part by Guangdong Province Key Laboratory of Popular High Performance Computers (2017B030314073), in part by Guangdong Province Engineering Center of China-made High Performance Data Computing System and Shenzhen Key Laboratory of Service Computing and Applications. Yi Wang is the corresponding author.

REFERENCES

- [1] “CACTI,” <https://www.hpl.hp.com/research/cacti>, 2024.
- [2] “GPT-2 Large,” <https://huggingface.co/openai-community/gpt2-large>, 2024.
- [3] “LM-Eval,” <https://github.com/EleutherAI/lm-evaluation-harness>, 2024.
- [4] “The Pile,” <https://pile.eleuther.ai>, 2024.
- [5] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. De Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. Rae, E. Elsen, and L. Sifre, “Improving Language Models by Retrieving from Trillions of Tokens,” in *International Conference on Machine Learning (ICML)*, 2022, pp. 2206–2240.
- [6] J. Chen, Z. Zhong, K. Sun, C. Ma, R. Mao, and Y. Wang, “Lift: Exploiting Hybrid Stacked Memory for Energy-Efficient Processing of Graph Convolutional Networks,” in *Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [7] Z. Chen, L. Liang, Q. Liu, Z. Li, F. Zhang, Y. Lu, and Z. Gu, “A High-Throughput Private Inference Engine Based on 3D Stacked Memory,” in *Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [8] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding,” in *North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.
- [9] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” in *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2021, pp. 874–880.
- [10] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, “Atlas: Few-shot Learning with Retrieval Augmented Language Models,” *Journal of Machine Learning Research (JMLR)*, vol. 24, no. 251, pp. 1–43, 2023.
- [11] W. Jiang, M. Zeller, R. Waleffe, T. Hoefler, and G. Alonso, “Chameleon: A Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models,” in *International Workshop on Heterogeneous High-Performance Reconfigurable Computing (H2RC)*, 2023, pp. 1–15.
- [12] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data (TBD)*, vol. 7, no. 3, pp. 535–547, 2019.
- [13] H. Jégou, M. Douze, and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, pp. 117–128, 2011.
- [14] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product,” in *International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.
- [15] Y. Lee, H. Choi, S. Min, H. Lee, S. Beak, D. Jeong, J. W. Lee, and T. J. Ham, “ANNA: Specialized Architecture for Approximate Nearest Neighbor Search,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 169–183.
- [16] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator,” *Computer Architecture Letters (CAL)*, vol. 19, no. 2, pp. 106–109, 2020.
- [17] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs,” *Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 42, no. 4, pp. 824–836, 2020.
- [18] M. Poremba, I. Akgun, J. Yin, O. Kayiran, Y. Xie, and G. H. Loh, “There and Back Again: Optimizing the Interconnect in Networks of Memory Cubes,” in *International Symposium on Computer Architecture (ISCA)*, 2017, pp. 678–690.
- [19] N. Reimers and I. Gurevych, “Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4512–4525.
- [20] W. Shi, S. Min, M. Lomeli, C. Zhou, M. Li, X. V. Lin, N. A. Smith, L. Zettlemoyer, W. tau Yih, and M. Lewis, “In-Context Pretraining: Language Modeling Beyond Document Boundaries,” in *International Conference on Learning Representations (ICLR)*, 2024, pp. 1–16.
- [21] H. Sun, Z. Zhu, C. Wang, X. Ning, G. Dai, H. Yang, and Y. Wang, “Gibbon: An Efficient Co-Exploration Framework of NN Model and Processing-In-Memory Architecture,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, pp. 4075–4089, 2023.
- [22] S. Wang, Y. Xu, Y. Fang, Y. Liu, S. Sun, R. Xu, C. Zhu, and M. Zeng, “Training Data is More Valuable than You Think: A Simple and Effective Method by Retrieving from Training Data,” in *Association for Computational Linguistics (ACL)*, 2022, pp. 3170–3179.
- [23] W. Wang, L. Dong, H. Cheng, H. Song, X. Liu, X. Yan, J. Gao, and F. Wei, “Visually-Augmented Language Modeling,” in *International Conference on Learning Representations (ICLR)*, 2023, pp. 1–17.
- [24] Y. Wang, H. Liu, J. Yuan, J. Chen, T. Wang, C. Ma, and R. Mao, “Leanor: A Learning-Based Accelerator for Efficient Approximate Nearest Neighbor Search via Reduced Memory Access,” in *Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [25] S. Wu, Y. Xiong, Y. Cui, X. Liu, B. Tang, T.-W. Kuo, and C. J. Xue, “Improving Natural Language Understanding with Computation-Efficient Retrieval Augmentation,” in *International Conference on Learning Representations (ICLR)*, 2024, pp. 1–14.
- [26] B. Zheng, X. Zhao, L. Weng, Q. V. H. Nguyen, H. Liu, and C. S. Jensen, “PM-LSH: A Fast and Accurate In-Memory Framework for High-Dimensional Approximate NN and Closest Pair Search,” *The VLDB Journal (VLDBJ)*, vol. 31, no. 6, pp. 1339–1363, 2022.
- [27] Z. Zhong, T. Lei, and D. Chen, “Training Language Models with Memory Augmentation,” in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022, pp. 5657–5673.
- [28] Y. Zhu, Z. Zhu, G. Dai, F. Tu, H. Sun, K.-T. Cheng, H. Yang, and Y. Wang, “PIM-HLS: An Automatic Hardware Generation Tool for Heterogeneous Processing-In-Memory-based Neural Network Accelerators,” in *Design Automation Conference (DAC)*, 2023, pp. 1–6.